BCSE303L - Operating Systems



Module 3 – Scheduling

Module:3	Scheduling	9 hours
Processes Scheduling - CPU Scheduling: Pre-emptive, non-pre-emptive - Multiprocessor scheduling – Deadlocks - Resource allocation and management - Deadlock handling mechanisms: prevention, avoidance, detection, recovery.		

Dr. P.Keerthika

Associate Professor Senior

School of Computer Science and Engineering

Vellore Institute of Technology, Vellore

Deadlock

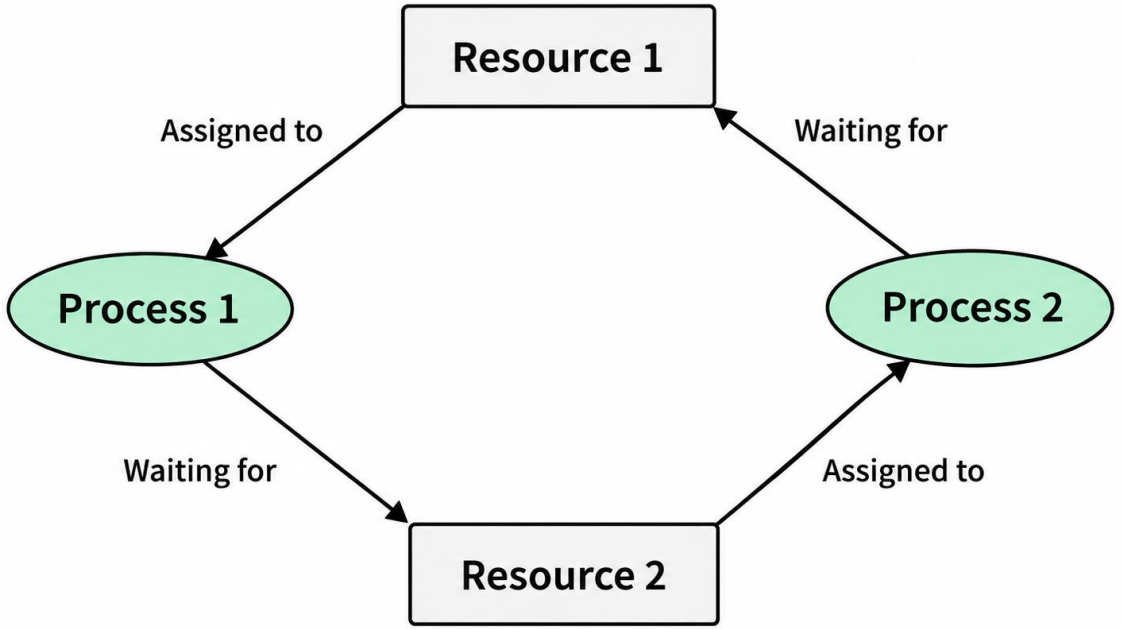
- Deadlock is a state in an operating system where two or more processes are stuck forever because each is waiting for a resource held by another.
- It happens only when four conditions exist:
  - mutual exclusion
  - hold and wait
  - no preemption
  - circular wait.
- For example, P1 holds R1 and needs R2, while P2 holds R2 and needs R1, so both wait forever.
- Deadlock can be handled using
  - Prevention
  - avoidance (Banker's algorithm)
  - detection and recovery

Who moves first? The answer is — **no one**.
That's a **deadlock**.

System Model

- System consists of resources
- Resource types $R_1, R_2, \ldots, R_m$ - CPU cycles, memory space, I/O devices
- Each resource type R_i has W_i instances.
- A process in an operating system typically uses resources in the following sequence:
  - Request a resource
  - Use the resource
  - Release the resource

Deadlock arises when processes hold some resources while waiting for others.



Deadlock - Example

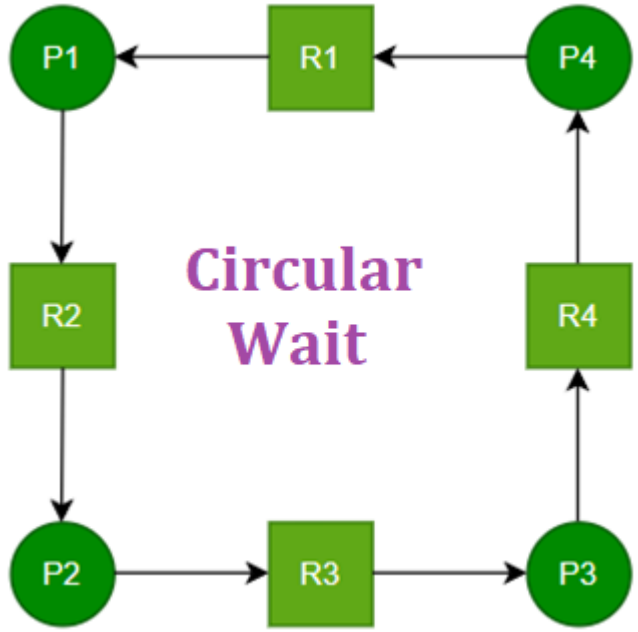
Example	Situation	Why deadlock occurs
Printer & Scanner	Process P1 holds the printer and waits for the scanner. P2 holds the scanner and waits for the printer.	Each waits for a resource held by the other.
Database Transactions	Transaction T1 locks Row A and requests Row B. T2 locks Row B and requests Row A.	Neither transaction can acquire its second lock.
Two Mutex Locks	Thread T1 locks Mutex A, then waits for Mutex B. Thread T2 locks Mutex B, then waits for Mutex A.	Circular lock dependency occurs.
File Locks	Program P1 locks File A and requests File B. P2 locks File B and requests File A.	Both programs wait indefinitely.
Traffic Intersection	Four cars enter an intersection from four directions, and each waits for the next car to move.	Every car blocks another car, forming a circular wait.
Dining Philosophers	Each philosopher picks up one fork and waits for the second fork held by the neighboring philosopher.	Everyone holds one resource while waiting for another.

Deadlock Characterization

- Deadlock can arise if the following four conditions hold simultaneously

(Necessary Conditions)

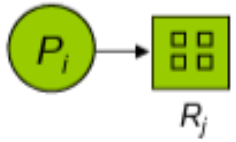
  - **Mutual Exclusion:** Only one process can use a resource at any given time i.e. the resources are non-sharable.
  - **Hold and Wait:** A process is holding at least one resource at a time and is waiting to acquire other resources held by some other process.
  - **No Preemption:** A resource cannot be taken from a process unless the process releases the resource.
  - **Circular Wait:** set of processes are waiting for each other in a circular fashion.

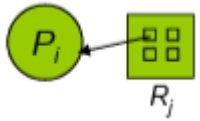


Resource Allocation Graph

- When processes request and hold resources, those relationships can be drawn as a ***graph*** **-** Visualizing the situation makes it easy to see when a deadlock can happen.

Components of the Resource-Allocation Graph (RAG):

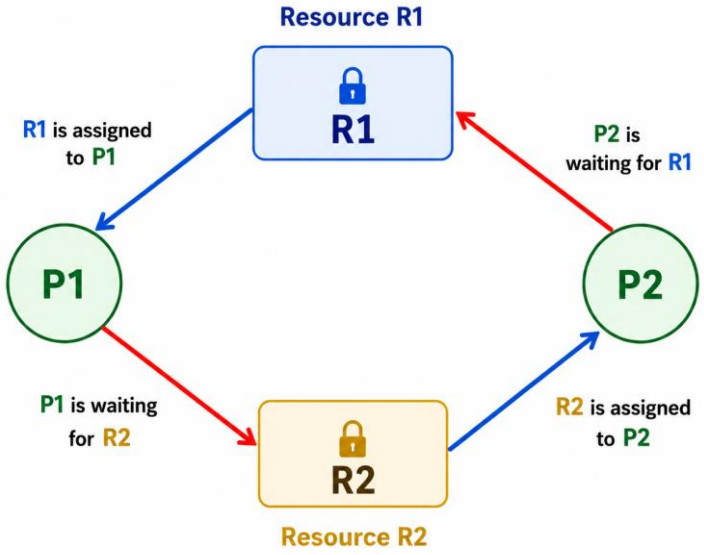
- **Process nodes**: usually drawn as circles labeled **P1, P2, ….** 
- **Resource nodes**: usually drawn as squares or rectangles labeled **R1, R2, ….** If a resource has **multiple identical instances**, it's common to show that number inside the resource box (e.g., **R1 (2)**).  **(Resource Type with 4 instances)**
- **Request edge**: a directed edge Pi → Rj means process Pi has requested resource Rj and is waiting. 

(Pi requests instance of Rj)
- **Assignment edge**: a directed edge Rj → Pi means resource Rj is currently assigned to Pi. 

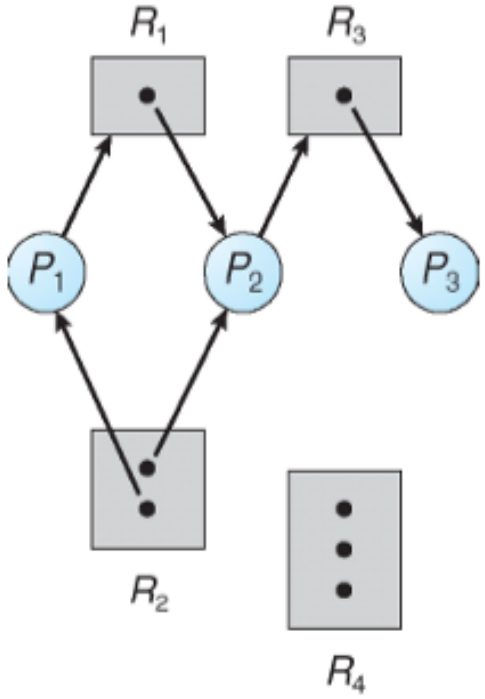
Pi is holding an instance of Rj)

Resource Allocation Graph

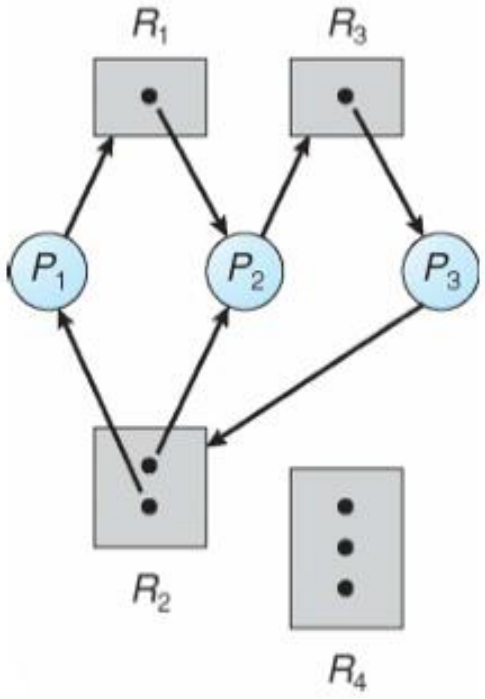
A resource-allocation graph shows *who holds what* and *who is waiting for what* at a glance



Single instance Resource Type With Deadlock



Multiple instances Resource Type Without Deadlock



Multiple instances Resource Type With Deadlock

Fact:

If graph **contains** no cycles
⇒ **no deadlock**
If graph **contains a cycle**
⇒

if only one instance per resource type, then deadlock

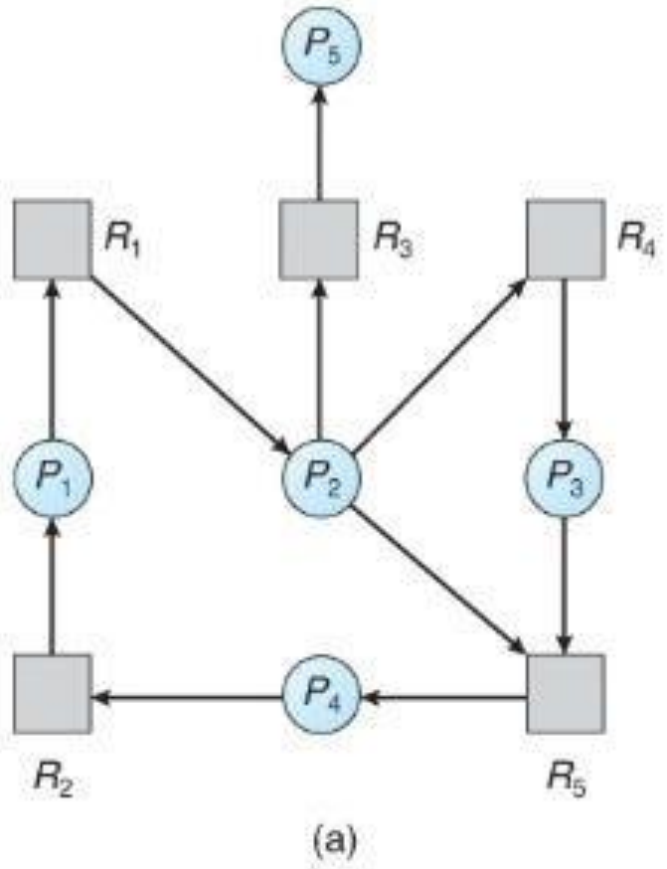
if several instances per resource type, possibility of deadlock

Wait - for Graph

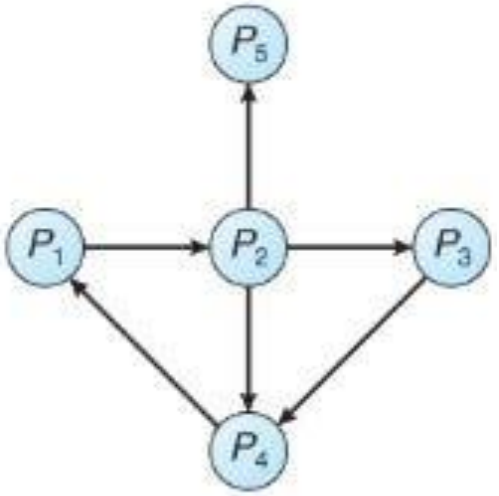
- The wait-for graph collapses resources out of the picture and draws only processes.
- There is a directed edge Pi → Pj if Pi is waiting for a resource currently held by Pj.
- The wait-for graph is especially useful when each resource type has only one instance.

For single-instance resources - A **cycle in the wait-for graph** directly indicates a deadlock

For multiple-instance resources - A cycle might exist without deadlock, but it's still a warning sign



(a)



(b)

Deadlock Handling Strategies

- Ignore the Problem (The "Ostrich Approach")
  - System simply ignores deadlocks and assumes they are rare or unlikely.
  - Most general-purpose operating systems, like Windows or Linux, follow this approach because constantly checking for deadlocks adds unnecessary overhead.
  - Idea: "If it rarely happens, don't waste time preventing it."
- Deadlock Prevention
  - This strategy ensures that at least one of the four necessary conditions for deadlock can never occur.
  - By breaking a condition, the system makes deadlock impossible.
  - Examples:
    - Request all resources at once (removes hold and wait).
    - Impose an order on resource requests (prevents circular wait).
      - Simply assign each resource (i.e., mutex locks) a unique number.
      - Resources must be acquired in order.
    - Allow resource preemption (breaks no preemption).

Deadlock Handling Strategies

- Deadlock Avoidance
  - System dynamically examines the resource-allocation state and carefully decides whether to grant a resource request based on whether the system will remain in a safe state afterward.
  - A safe state means there exists some sequence in which all processes can finish without getting stuck; an unsafe state might lead to deadlock.
  - Example: The Banker's Algorithm checks every request to ensure the system won't enter an unsafe state.
- Deadlock Detection and Recovery
  - This approach lets deadlocks occur occasionally but includes mechanisms to detect and recover from them.
  - Detection algorithms look for cycles in the system's resource allocation graph, and recovery methods then free resources or terminate processes.
  - **Recovery options:**
    - Terminate one or more processes involved in the deadlock.
    - Preempt (take back) resources and restart affected processes.
    - Roll back processes to a safe point/state.

Deadlock Prevention

- Aims to eliminate at least one of the four necessary conditions for deadlock, ensuring that the system never enters a deadlock state.

1. **Eliminating Hold and Wait**
   - Allocate all required resources to the process before start of its execution – eliminates hold and wait condition - but it will lead to low device utilization.
2. **Allowing Resource Preemption**
   - Preempt resources from process when resources are required by other high priority processes
3. **Imposing Resource Ordering**
   - Assign a priority number to each of the resource.
   - A process can request for the resources only in increasing order of numbering (e.g., R1 < R2 < R3).
   - This ensures that not a single process can request a resource which is being utilized by some other process and no cycle will be formed.
4. **Disallowing Mutual Exclusion**
   - Where possible, resources are made shareable, allowing multiple processes to access them at the same time.
   - This prevents one process from exclusively locking a resource unnecessarily.

Deadlock Avoidance

Requires that the system has some additional a priori information available

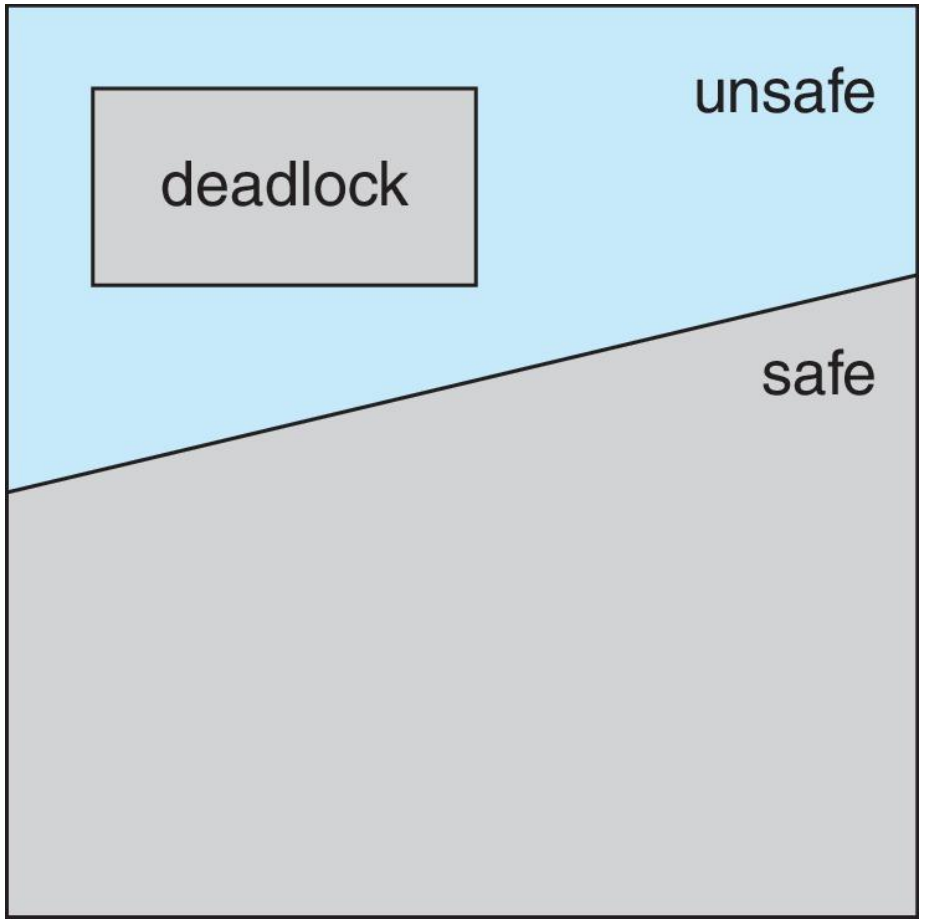
- Simplest and most useful model requires that each process declare the maximum number of resources of each type that it may need
- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition
- Resource-allocation state is defined by the number of available and allocated resources, and the maximum demands of the processes

Deadlock Avoidance

- **Safe State (Deadlock Avoidance)**
  - When a process requests an available resource, system must decide if immediate allocation leaves the system in a safe state
  - System is in **safe state** if there exists a sequence $<P_1, P_2, \ldots, P_n>$ of ALL the process in the systems such that
    - for each P_i, the resources that P_i can still request can be satisfied by currently available resources + resources held by all the P_j, with $j < I$
  - That is:
    - If P_i resource needs are not immediately available, then P_i can wait until all P_j have finished
    - When P_j is finished, P_i can obtain needed resources, execute, return allocated resources, and terminate
    - When P_i terminates, P_{i+1} can obtain its needed resources, and so on

Deadlock Avoidance

- **Safe State (Deadlock Avoidance)**
  - If a system is in safe state - no deadlocks
  - If a system is in unsafe state - possibility of deadlock
  - Avoidance - ensure that a system will never enter an unsafe state.



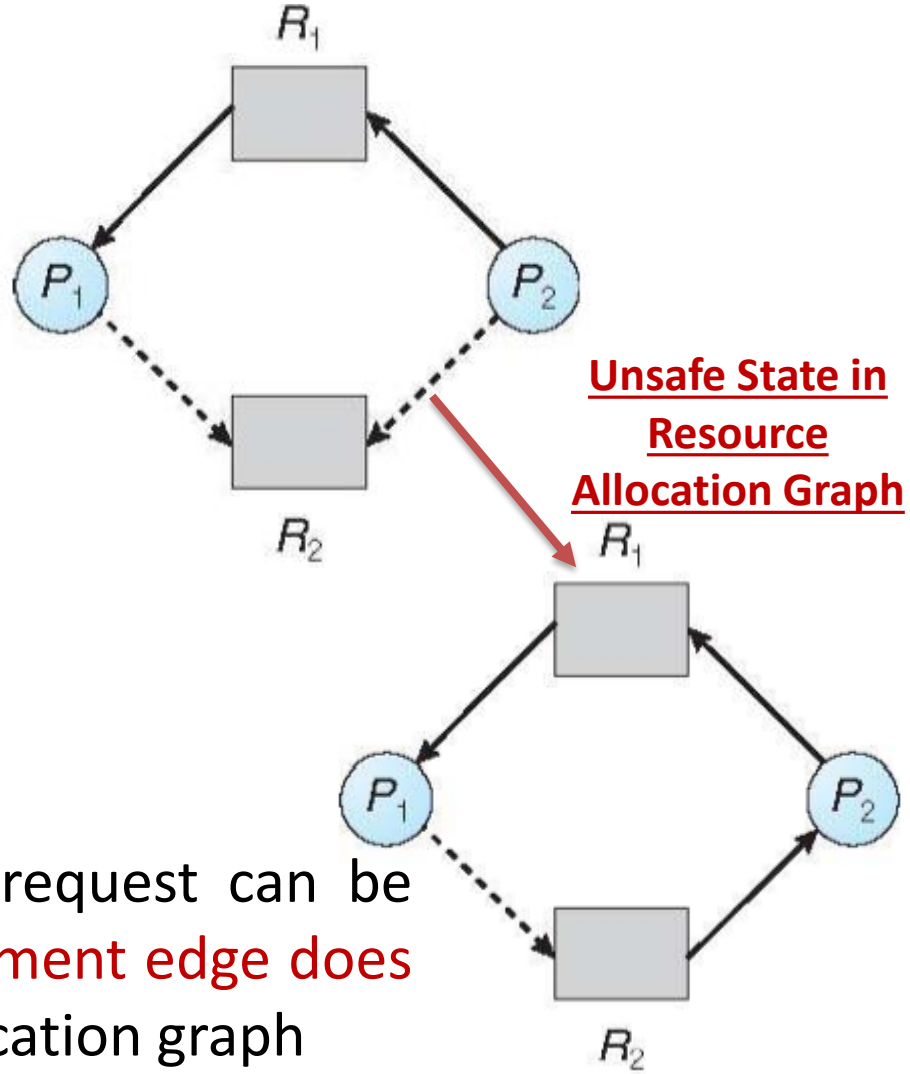
Safe, Unsafe, Deadlock State

Deadlock Avoidance Algorithms

- Single instance of a resource type
  - Use resource-allocation graph Algorithm
- Multiple instances of a resource type
  - Use the Banker's Algorithm

Deadlock Avoidance Algorithms - Resource-allocation graph Scheme

- Claim edge Pi → Rj indicated that process Pi may request resource Rj ; represented by a dashed line
- Claim edge converts to Request edge when a process requests a resource
- Request edge converted to an assignment edge when the resource is allocated to the process
- When a resource is released by a process, assignment edge reconverts to a claim edge
- Resources must be claimed a priori in the system



Resource-allocation graph Algorithm

- Suppose that process Pi requests a resource Rj. The request can be granted only if converting the request edge to an assignment edge does not result in the formation of a cycle in the resource allocation graph

Deadlock Avoidance Algorithms

Banker's Algorithm

- for Multiple instance of a resource type
- Each process must make a priori claim for maximum use
- When a process requests a resource it may have to wait
- When a process gets all its resources it must return them in a finite amount of time
- Has
  - Safety Algorithm
  - Resource-Request Algorithm

Deadlock Avoidance Algorithms

Banker's Algorithm

Let n = number of processes and m = number of resources types.

- **Available:** Vector of length m.
  - If available [j] = k, there are k instances of resource type Rj available
- **Max:** n x m matrix.
  - If Max [i,j] = k, then process Pi may request at most k instances of resource type Rj
- **Allocation:** n x m matrix.
  - If Allocation[i,j] = k then Pi is currently allocated k instances of Rj
- **Need:** n x m matrix.
  - If Need[i,j] = k, then Pi may need k more instances of Rj to complete its task
  - ***Need [i,j] = Max[i,j] – Allocation [i,j]***

Deadlock Avoidance Algorithms

Safety Algorithm

Banker's Algorithm

To check whether a system is currently in safe state

1. Let ***Work*** and ***Finish*** be vectors of length *m* and *n*, respectively.
   Initialize:
   Work* = *Available
   ***Finish* [*i*] = *false* for *i* = 0, 1, …, *n*- 1**

2. Find an ***i*** such that both:
   (a) ***Finish* [*i*] = *false***
   (b) $Need_i \leq Work$
   If no such ***i*** exists - go to step 4

3. $Work = Work + Allocation_i$
   Finish*[*i*] = *true
   go to step 2

4. If ***Finish* [*i*] == *true*** for all ***i***, then the system is in a safe state

Deadlock Avoidance Algorithms

Resource Request Algorithm

Banker's Algorithm

To check whether a Resource Request can be granted in future without any deadlock

$Request_i$ = request vector for process T_i.

If $Request_i[j] = k$ then process T_i wants k instances of resource type R_j

1. If $Request_i \leq Need_i$ go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If $Request_i \leq Available$, go to step 3. Otherwise T_i must wait, since resources are not available
3. Pretend to allocate requested resources to T_i by modifying the state as follows:

$$Available = Available - Request_i;$$
$$Allocation_i = Allocation_i + Request_i;$$
$$Need_i = Need_i - Request_i;$$

- If safe $\Rightarrow$ the resources are allocated to T_i
- If unsafe $\Rightarrow$ T_i must wait, and the old resource-allocation state is restored

Deadlock Avoidance Algorithms

Example 1

Banker's Algorithm

1. Number of instances of each resource a process can max request – MAX
2. Number of instances of each resource a process currently holds – ALLOCATION
3. Number of instances of each resource available in the system – AVAILABLE

PROCESS	ALLOCATION			
	A	B	C	D
P0	0	0	1	2
P1	1	0	0	0
P2	1	3	5	4
P3	0	6	3	2
P4	0	0	1	4

MAX			
A	B	C	D
0	0	1	2
1	7	5	0
2	3	5	6
0	6	5	2
0	6	5	6

AVAILABLE			
A	B	C	D
1	5	2	0

Deadlock Avoidance Algorithms

Example 1

Banker's Algorithm

PROCESS	ALLOCATION			
	A	B	C	D
P0	0	0	1	2
P1	1	0	0	0
P2	1	3	5	4
P3	0	6	3	2
P4	0	0	1	4

MAX			
A	B	C	D
0	0	1	2
1	7	5	0
2	3	5	6
0	6	5	2
0	6	5	6

AVAILABLE			
A	B	C	D
1	5	2	0

Steps:

1. Calculate NEED Matrix
2. Is system in safe state?

   If yes, then, find safe sequence

Deadlock Avoidance Algorithms

Example 1

Banker's Algorithm

PROCESS	ALLOCATION			
	A	B	C	D
P0	0	0	1	2
P1	1	0	0	0
P2	1	3	5	4
P3	0	6	3	2
P4	0	0	1	4

MAX			
A	B	C	D
0	0	1	2
1	7	5	0
2	3	5	6
0	6	5	2
0	6	5	6

AVAILABLE			
A	B	C	D
1	5	2	0

Steps:

1. Calculate NEED Matrix
2. Is system in safe state?

   If yes, then, find safe sequence

Total No. of Instances: (Allocated + Available)

A	B	C	D
3	14	12	12

Deadlock Avoidance Algorithms

Example 1

Banker's Algorithm

PROCESS	ALLOCATION			
	A	B	C	D
P0	0	0	1	2
P1	1	0	0	0
P2	1	3	5	4
P3	0	6	3	2
P4	0	0	1	4

MAX			
A	B	C	D
0	0	1	2
1	7	5	0
2	3	5	6
0	6	5	2
0	6	5	6

AVAILABLE			
A	B	C	D
1	5	2	0

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	NEED			
	A	B	C	D
P0	0	0	0	0
P1	0	7	5	0
P2	1	0	0	2
P3	0	0	2	0
P4	0	6	4	2

Steps:

1. Calculate NEED Matrix
2. Is system in safe state?

   If yes, then, find safe sequence

Total No. of Instances: (Allocated + Available)

A	B	C	D
3	14	12	12

Now, check, whether the system is in safe state?

Deadlock Avoidance Algorithms

Example 1

Banker's Algorithm

PROCESS	ALLOCATION			
	A	B	C	D
P0	~~0~~	~~0~~	~~1~~	~~2~~
P1	1	0	0	0
P2	1	3	5	4
P3	0	6	3	2
P4	0	0	1	4

MAX			
A	B	C	D
0	0	1	2
1	7	5	0
2	3	5	6
0	6	5	2
0	6	5	6

AVAILABLE			
A	B	C	D
1	5	2	0
1	5	3	2

NEED Matrix

NEED = MAX_i - $ALLOC_i$

PROCESS	NEED			
	A	B	C	D
P0	0	0	0	0
P1	0	7	5	0
P2	1	0	0	2
P3	0	0	2	0
P4	0	6	4	2

1. *Compare NEED and AVAILABLE*
2. *AVAILABLE = AVAILABLE + ALLOCATED*

Safe Sequence:

<P0,

$Available = Available - Request_i$;
$Allocation_i = Allocation_i + Request_i$;
$Need_i = Need_i - Request_i$;

Now, check, whether the system is in safe state?

Deadlock Avoidance Algorithms

Example 1

Banker's Algorithm

PROCESS	ALLOCATION			
	A	B	C	D
P0	~~0~~	~~0~~	~~1~~	~~2~~
P1	1	0	0	0
P2	~~1~~	~~3~~	~~5~~	~~4~~
P3	0	6	3	2
P4	0	0	1	4

MAX			
A	B	C	D
0	0	1	2
1	7	5	0
2	3	5	6
0	6	5	2
0	6	5	6

AVAILABLE			
A	B	C	D
1	5	2	0
1	5	3	2
2	8	8	6

NEED Matrix

NEED = MAX_i - $ALLOC_i$

PROCESS	NEED			
	A	B	C	D
P0	0	0	0	0 ✔
P1	0	7	5	0
P2	1	0	0	2 ✔
P3	0	0	2	0
P4	0	6	4	2

1. *Compare NEED and AVAILABLE*
2. *AVAILABLE = AVAILABLE + ALLOCATED*

Safe Sequence:

<P0, P2,

$Available = Available - Request_i$;
$Allocation_i = Allocation_i + Request_i$;
$Need_i = Need_i - Request_i$;

Now, check, whether the system is in safe state?

Deadlock Avoidance Algorithms

Banker's Algorithm

Example 1

PROCESS	ALLOCATION			
	A	B	C	D
P0	~~0~~	~~0~~	~~1~~	~~2~~
P1	1	0	0	0
P2	~~1~~	~~3~~	~~5~~	~~4~~
P3	~~0~~	~~6~~	~~3~~	~~2~~
P4	0	0	1	4

MAX			
A	B	C	D
0	0	1	2
1	7	5	0
2	3	5	6
0	6	5	2
0	6	5	6

AVAILABLE			
A	B	C	D
1	5	2	0
1	5	3	2
2	8	8	6
2	14	11	8

NEED Matrix

NEED = MAX_i - $ALLOC_i$

PROCESS	NEED			
	A	B	C	D
P0	0	0	0	0 ✔
P1	0	7	5	0
P2	1	0	0	2 ✔
P3	0	0	2	0 ✔
P4	0	6	4	2

1. *Compare NEED and AVAILABLE*
2. *AVAILABLE = AVAILABLE + ALLOCATED*

Safe Sequence:

<P0, P2, P3

$Available = Available - Request_i$;
$Allocation_i = Allocation_i + Request_i$;
$Need_i = Need_i - Request_i$;

Now, check, whether the system is in safe state?

Deadlock Avoidance Algorithms

Example 1

Banker's Algorithm

PROCESS	ALLOCATION			
	A	B	C	D
P0	~~0~~	~~0~~	~~1~~	~~2~~
P1	1	0	0	0
P2	~~1~~	~~3~~	~~5~~	~~4~~
P3	~~0~~	~~6~~	~~3~~	~~2~~
P4	~~0~~	~~0~~	~~1~~	~~4~~

MAX			
A	B	C	D
0	0	1	2
1	7	5	0
2	3	5	6
0	6	5	2
0	6	5	6

AVAILABLE			
A	B	C	D
1	5	2	0
1	5	3	2
2	8	8	6
2	14	11	8
2	14	12	12

NEED Matrix

NEED = MAX_i - $ALLOC_i$

PROCESS	NEED			
	A	B	C	D
P0	0	0	0	0 ✔
P1	0	7	5	0
P2	1	0	0	2 ✔
P3	0	0	2	0 ✔
P4	0	6	4	2 ✔

1. *Compare NEED and AVAILABLE*
2. *AVAILABLE = AVAILABLE + ALLOCATED*

Safe Sequence:

<P0, P2, P3, P4

$Available = Available - Request_i$;
$Allocation_i = Allocation_i + Request_i$;
$Need_i = Need_i - Request_i$;

Now, check, whether the system is in safe state?

Deadlock Avoidance Algorithms

Example 1

Banker's Algorithm

PROCESS	ALLOCATION			
	A	B	C	D
P0	~~0~~	~~0~~	~~1~~	~~2~~
P1	~~1~~	~~0~~	~~0~~	~~0~~
P2	~~1~~	~~3~~	~~5~~	~~4~~
P3	~~0~~	~~6~~	~~3~~	~~2~~
P4	~~0~~	~~0~~	~~1~~	~~4~~

MAX			
A	B	C	D
0	0	1	2
1	7	5	0
2	3	5	6
0	6	5	2
0	6	5	6

AVAILABLE			
A	B	C	D
1	5	2	0
1	5	3	2
2	8	8	6
2	14	11	8
2	14	12	12
3	14	12	12

NEED Matrix

NEED = MAX_i - $ALLOC_i$

PROCESS	NEED			
	A	B	C	D
P0	0	0	0	0 ✔
P1	0	7	5	0 ✔
P2	1	0	0	2 ✔
P3	0	0	2	0 ✔
P4	0	6	4	2 ✔

1. *Compare NEED and AVAILABLE*
2. *AVAILABLE = AVAILABLE + ALLOCATED*

Safe Sequence:

<P0, P2, P3, P4, P1 >

Available = Available – $Request_i$;
$Allocation_i$ = $Allocation_i$ + $Request_i$;
$Need_i$ = $Need_i$ – $Request_i$;

Now, check, whether the system is in safe state?

Deadlock Avoidance Algorithms

Example 1

Banker's Algorithm

PROCESS	ALLOCATION			
	A	B	C	D
P0	~~0~~	~~0~~	~~1~~	~~2~~
P1	~~1~~	~~0~~	~~0~~	~~0~~
P2	~~1~~	~~3~~	~~5~~	~~4~~
P3	~~0~~	~~6~~	~~3~~	~~2~~
P4	~~0~~	~~0~~	~~1~~	~~4~~

MAX			
A	B	C	D
0	0	1	2
1	7	5	0
2	3	5	6
0	6	5	2
0	6	5	6

AVAILABLE			
A	B	C	D
1	5	2	0
1	5	3	2
2	8	8	6
2	14	11	8
2	14	12	12
3	14	12	12

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	NEED			
	A	B	C	D
P0	0	0	0	0 ✔
P1	0	7	5	0 ✔
P2	1	0	0	2 ✔
P3	0	0	2	0 ✔
P4	0	6	4	2 ✔

1. *Compare NEED and AVAILABLE*
2. *AVAILABLE = AVAILABLE + ALLOCATED*

Safe Sequence:

<P0, P2, P3, P4, P1 >

Total No. of Instances: (Allocated + Available)

A	B	C	D
3	14	12	12

$Available = Available - Request_i$;
$Allocation_i = Allocation_i + Request_i$;
$Need_i = Need_i - Request_i$;

Now, check, whether the system is in safe state?

Deadlock Avoidance Algorithms

Safety Algorithm

Banker's Algorithm

To check whether a system is currently in safe state

Input: Processes

- Any 2 out of 3 (MAX, NEED, ALLOCATION)
- Any 1 out of 2 (Available, Total No. of Resources)

N = No. of Processes

M = No. Resources

Step 1: set flag[i]=0 for i=0 to n-1

find *NEED[n][m] = MAX [n][m] – ALLOCATION [n][m]*

Step 2: Find a Process Pi such that flag[i] = 0 and $NEED_i$ <=AVAILABLE

Step 3: If such Pi exists, then,

flag[i] =1,

AVAILABLE = AVAILABLE + $ALLOCATION_i$

goto step2

else goto step 4.

Step 4: if flag[i] =1, for all i,

then, System in safe state

else Unsafe

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

PROCESS	ALLOCATION			
	A	B	C	D
P0	2	0	0	1
P1	3	1	2	1
P2	2	1	0	3
P3	1	3	1	2
P4	1	4	3	2

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
3	3	2	1

1. Calculate NEED Matrix
2. Is the System in safe state? If so, find the safe sequence.
3. If request from P1 arrives for (1,1,0,0). Can request be immediately granted?
4. If request from P4 arrives for (0,0,2,0). Can it be immediately granted?

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	2	0	0	1
P1	3	1	2	1
P2	2	1	0	3
P3	1	3	1	2
P4	1	4	3	2

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
3	3	2	1

PROCESS	NEED			
	A	B	C	D
P0	2	2	1	1
P1	2	1	3	1
P2	0	2	1	3
P3	0	1	1	2
P4	2	2	3	3

1. Calculate NEED Matrix

Total No. of Instances:
(Allocated + Available)

A	B	C	D
12	12	8	10

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	~~2~~	~~0~~	~~0~~	~~1~~
P1	3	1	2	1
P2	2	1	0	3
P3	1	3	1	2
P4	1	4	3	2

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
3	3	2	1
5	3	**2**	**2**

PROCESS	NEED			
	A	B	C	D
P0	2	2	1	1
P1	2	1	3	1
P2	0	2	1	3
P3	0	1	1	2
P4	2	2	3	3

✔

2. Is the System in safe state? If so, find the safe sequence.

Total No. of Instances: (Allocated + Available)

A	B	C	D
12	12	8	10

Safe Sequence: <P0,

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	~~2~~	~~0~~	~~0~~	~~1~~
P1	3	1	2	1
P2	2	1	0	3
P3	~~1~~	~~3~~	~~1~~	~~2~~
P4	1	4	3	2

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
3	3	2	1
5	3	2	2
6	6	3	4

PROCESS	NEED			
	A	B	C	D
P0	2	2	1	1 ✔
P1	2	1	3	1
P2	0	2	1	3
P3	0	1	1	2 ✔
P4	2	2	3	3

2. Is the System in safe state? If so, find the safe sequence.

Total No. of Instances: (Allocated + Available)

A	B	C	D
12	12	8	10

Safe Sequence: <P0, P3,

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	~~2~~	~~0~~	~~0~~	~~1~~
P1	~~3~~	~~1~~	~~2~~	~~1~~
P2	2	1	0	3
P3	~~1~~	~~3~~	~~1~~	~~2~~
P4	1	4	3	2

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
3	3	2	1
5	3	**2**	2
6	6	3	4
9	7	5	5

PROCESS	NEED			
	A	B	C	D
P0	2	2	1	1 ✔
P1	2	1	3	1 ✔
P2	0	2	1	3
P3	0	1	1	2 ✔
P4	2	2	3	3

2. Is the System in safe state? If so, find the safe sequence.

Total No. of Instances: (Allocated + Available)

A	B	C	D
12	12	8	10

Safe Sequence: <P0, P3, P1,

Other Safe Sequence:

<P0, P3, P4, P1, P2>

<P0, P3, P2,P1, P4>

<P0, P3, P2, P4, P1>

.....

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	~~2~~	~~0~~	~~0~~	~~1~~
P1	~~3~~	~~1~~	~~2~~	~~1~~
P2	~~2~~	~~1~~	~~0~~	~~3~~
P3	~~1~~	~~3~~	~~1~~	~~2~~
P4	1	4	3	2

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
3	3	2	1
5	3	2	2
6	6	3	4
9	7	5	5
11	8	5	8

PROCESS	NEED			
	A	B	C	D
P0	2	2	1	1 ✔
P1	2	1	3	1 ✔
P2	0	2	1	3 ✔
P3	0	1	1	2 ✔
P4	2	2	3	3

2. Is the System in safe state? If so, find the safe sequence.

Total No. of Instances: (Allocated + Available)

A	B	C	D
12	12	8	10

Safe Sequence: <P0, P3, P1, P2

Other Safe Sequence:

<P0, P3, P4, P1, P2>

<P0, P3, P2,P1, P4>

<P0, P3, P2, P4, P1>

.....

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	~~2~~	~~0~~	~~0~~	~~1~~
P1	~~3~~	~~1~~	~~2~~	~~1~~
P2	~~2~~	~~1~~	~~0~~	~~3~~
P3	~~1~~	~~3~~	~~1~~	~~2~~
P4	~~1~~	~~4~~	~~3~~	~~2~~

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
3	3	2	1
5	3	2	2
6	6	3	4
9	7	5	5
11	8	5	8
12	12	8	10

PROCESS	NEED			
	A	B	C	D
P0	2	2	1	1 ✔
P1	2	1	3	1 ✔
P2	0	2	1	3 ✔
P3	0	1	1	2 ✔
P4	2	2	3	3 ✔

2. Is the System in safe state? If so, find the safe sequence.

Total No. of Instances: (Allocated + Available)

A	B	C	D
12	12	8	10

Safe Sequence: <P0, P3, P1, P2, P4>

Other Safe Sequence:

<P0, P3, P4, P1, P2>

<P0, P3, P2,P1, P4>

<P0, P3, P2, P4, P1>

…….

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	2	0	0	1
P1	3	1	2	1
P2	2	1	0	3
P3	1	3	1	2
P4	1	4	3	2

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
3	3	2	1

PROCESS	NEED			
	A	B	C	D
P0	2	2	1	1
P1	2	1	3	1
P2	0	2	1	3
P3	0	1	1	2
P4	2	2	3	3

3. If request from P1 arrives for (1,1,0,0). Can request be immediately granted?

Uses Resource Request Algorithm

Deadlock Avoidance Algorithms

Resource Request Algorithm

Banker's Algorithm

To check whether a Resource Request can be granted in future without any deadlock

$Request_i$ = request vector for process T_i.

If $Request_i[j] = k$ then process T_i wants k instances of resource type R_j

1. If $Request_i \leq Need_i$ go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If $Request_i \leq Available$, go to step 3. Otherwise T_i must wait, since resources are not available
3. Pretend to allocate requested resources to T_i by modifying the state as follows:

$$Available = Available - Request_i;$$
$$Allocation_i = Allocation_i + Request_i;$$
$$Need_i = Need_i - Request_i;$$

- If safe $\Rightarrow$ the resources are allocated to T_i
- If unsafe $\Rightarrow$ T_i must wait, and the old resource-allocation state is restored

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	2	0	0	1
P1	3	1	2	1
P2	2	1	0	3
P3	1	3	1	2
P4	1	4	3	2

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
3	3	2	1

PROCESS	NEED			
	A	B	C	D
P0	2	2	1	1
P1	2	1	3	1
P2	0	2	1	3
P3	0	1	1	2
P4	2	2	3	3

3. If request from P1 arrives for (1,1,0,0). Can request be immediately granted?

Uses Resource Request Algorithm

Step 1: Check **REQUEST(P1) <NEED(P1)** → (1,1,0,0) < (2,1,3,1) → Yes

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	2	0	0	1
P1	3	1	2	1
P2	2	1	0	3
P3	1	3	1	2
P4	1	4	3	2

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
3	3	2	1

PROCESS	NEED			
	A	B	C	D
P0	2	2	1	1
P1	2	1	3	1
P2	0	2	1	3
P3	0	1	1	2
P4	2	2	3	3

3. If request from P1 arrives for (1,1,0,0). Can request be immediately granted?

Uses Resource Request Algorithm

Step 1: Check **REQUEST(P1) <NEED(P1)** → (1,1,0,0) < (2,1,3,1) → **Yes**

Step 2: Check **REQUEST(P1) < =AVAILABLE** → (1,1,0,0) < = (3,3,2,1) → **Yes**

So, the request from P1 for (1,1,0,0) can be granted

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	2	0	0	1
P1	3	1	2	1
P2	2	1	0	3
P3	1	3	1	2
P4	1	4	3	2

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
3	3	2	1

PROCESS	NEED			
	A	B	C	D
P0	2	2	1	1
P1	2	1	3	1
P2	0	2	1	3
P3	0	1	1	2
P4	2	2	3	3

3. If request from P1 arrives for (1,1,0,0). Can request be immediately granted?

Uses Resource Request Algorithm

Step 1: Check **REQUEST(P1) <NEED(P1)** → (1,1,0,0) < (2,1,3,1) → Yes

Step 2: Check **REQUEST(P1) < =AVAILABLE** → (1,1,0,0) < = (3,3,2,1) → Yes

So, the request from P1 for (1,1,0,0) can be granted

Step 3: But, the System will not immediately allocate the resource. Rather, it pretends that the resource is allocated and check for safe state (no deadlock) if allocated. **(Uses Safety Algorithm)**

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	2	0	0	1
P1	3	1	2	1
P2	2	1	0	3
P3	1	3	1	2
P4	1	4	3	2

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
3	3	2	1

PROCESS	NEED			
	A	B	C	D
P0	2	2	1	1
P1	2	1	3	1
P2	0	2	1	3
P3	0	1	1	2
P4	2	2	3	3

3. If request from P1 arrives for (1,1,0,0). Can request be immediately granted?

System pretends that the resource is allocated and check for safe state (no deadlock) if allocated. **(Uses Safety Algorithm)**

Update NEED (P1) = NEED (P1) – REQUEST (P1)

AVAILABLE = AVAILABLE – REQUEST (P1)

ALLOCATION (P1) = ALLOCATION (P1) + REQUEST (P1)

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	2	0	0	1
P1	3	1	2	1
P2	2	1	0	3
P3	1	3	1	2
P4	1	4	3	2

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
3	3	2	1

PROCESS	NEED			
	A	B	C	D
P0	2	2	1	1
P1	~~2~~ 1	~~1~~ 0	3	1
P2	0	2	1	3
P3	0	1	1	2
P4	2	2	3	3

(2,1,3,1) – (1,1,0,0) =(1,0,3,1)

3. If request from P1 arrives for (1,1,0,0). Can request be immediately granted?

System pretends that the resource is allocated and check for safe state (no deadlock) if allocated. **(Uses Safety Algorithm)**

Update NEED (P1) = NEED (P1) – REQUEST (P1)

AVAILABLE = AVAILABLE – REQUEST (P1)

ALLOCATION (P1) = ALLOCATION (P1) + REQUEST (P1)

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	2	0	0	1
P1	3	1	2	1
P2	2	1	0	3
P3	1	3	1	2
P4	1	4	3	2

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
~~3~~ 2	~~3~~ 2	2	1

(3,3,2,1) – (1,1,0,0) =(2,2,2,1)

PROCESS	NEED			
	A	B	C	D
P0	2	2	1	1
P1	~~2~~ 1	~~1~~ 0	3	1
P2	0	2	1	3
P3	0	1	1	2
P4	2	2	3	3

3. If request from P1 arrives for (1,1,0,0). Can request be immediately granted?

System pretends that the resource is allocated and check for safe state (no deadlock) if allocated. **(Uses Safety Algorithm)**

Update NEED (P1) = NEED (P1) – REQUEST (P1)

AVAILABLE = AVAILABLE – REQUEST (P1)

ALLOCATION (P1) = ALLOCATION (P1) + REQUEST (P1)

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	2	0	0	1
P1	~~3~~ 4	~~1~~ 2	2	1
P2	2	1	0	3
P3	1	3	1	2
P4	1	4	3	2

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
~~3~~ 2	~~3~~ 2	2	1

(3,1,2,1) +
(1,1,0,0)
=(4,2,2,1)

PROCESS	NEED			
	A	B	C	D
P0	2	2	1	1
P1	~~2~~ 1	~~1~~ 0	3	1
P2	0	2	1	3
P3	0	1	1	2
P4	2	2	3	3

3. If request from P1 arrives for (1,1,0,0). Can request be immediately granted?

System pretends that the resource is allocated and check for safe state (no deadlock) if allocated. **(Uses Safety Algorithm)**

Update NEED (P1) = NEED (P1) – REQUEST (P1)
AVAILABLE = AVAILABLE – REQUEST (P1)
ALLOCATION (P1) = ALLOCATION (P1) + REQUEST (P1)

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	2	0	0	1
P1	4	2	2	1
P2	2	1	0	3
P3	1	3	1	2
P4	1	4	3	2

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
2	2	2	1

Updated System Snapshot

PROCESS	NEED			
	A	B	C	D
P0	2	2	1	1
P1	1	0	3	1
P2	0	2	1	3
P3	0	1	1	2
P4	2	2	3	3

3. If request from P1 arrives for (1,1,0,0). Can request be immediately granted?

System pretends that the resource is allocated and check for safe state (no deadlock) if allocated. **(Uses Safety Algorithm)**

Find Safe Sequence:

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	~~2~~	~~0~~	~~0~~	~~1~~
P1	4	2	2	1
P2	2	1	0	3
P3	1	3	1	2
P4	1	4	3	2

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
2	2	2	1
4	**2**	**2**	**2**

PROCESS	NEED			
	A	B	C	D
P0	2	2	1	1
P1	1	0	3	1
P2	0	2	1	3
P3	0	1	1	2
P4	2	2	3	3

✔

3. If request from P1 arrives for (1,1,0,0). Can request be immediately granted?

System pretends that the resource is allocated and check for safe state (no deadlock) if allocated. **(Uses Safety Algorithm)**

Find Safe Sequence:

Safe Sequence: <P0,

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	~~2~~	~~0~~	~~0~~	~~1~~
P1	4	2	2	1
P2	2	1	0	3
P3	~~1~~	~~3~~	~~1~~	~~2~~
P4	1	4	3	2

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
2	2	2	1
4	2	2	2
5	5	3	4

PROCESS	NEED			
	A	B	C	D
P0	2	2	1	1 ✔
P1	1	0	3	1
P2	0	2	1	3
P3	0	1	1	2 ✔
P4	2	2	3	3

3. If request from P1 arrives for (1,1,0,0). Can request be immediately granted?

System pretends that the resource is allocated and check for safe state (no deadlock) if allocated. **(Uses Safety Algorithm)**

Find Safe Sequence:

Safe Sequence: <P0, P3,

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	~~2~~	~~0~~	~~0~~	~~1~~
P1	4	2	2	1
P2	2	1	0	3
P3	~~1~~	~~3~~	~~1~~	~~2~~
P4	~~1~~	~~4~~	~~3~~	~~2~~

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
2	2	2	1
4	2	2	2
5	5	3	4
6	9	6	6

PROCESS	NEED				
	A	B	C	D	
P0	2	2	1	1	✔
P1	1	0	3	1	
P2	0	2	1	3	
P3	0	1	1	2	✔
P4	2	2	3	3	✔

3. If request from P1 arrives for (1,1,0,0). Can request be immediately granted?

System pretends that the resource is allocated and check for safe state (no deadlock) if allocated. **(Uses Safety Algorithm)**

Find Safe Sequence:

Safe Sequence: <P0, P3, P4,

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	~~2~~	~~0~~	~~0~~	~~1~~
P1	~~4~~	~~2~~	~~2~~	~~1~~
P2	2	1	0	3
P3	~~1~~	~~3~~	~~1~~	~~2~~
P4	~~1~~	~~4~~	~~3~~	~~2~~

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
2	2	2	1
4	2	2	2
5	5	3	4
6	9	6	6
10	11	8	7

PROCESS	NEED			
	A	B	C	D
P0	2	2	1	1 ✔
P1	1	0	3	1 ✔
P2	0	2	1	3
P3	0	1	1	2 ✔
P4	2	2	3	3 ✔

3. If request from P1 arrives for (1,1,0,0). Can request be immediately granted?

System pretends that the resource is allocated and check for safe state (no deadlock) if allocated. **(Uses Safety Algorithm)**

Find Safe Sequence:

Safe Sequence: <P0, P3, P4, P1,

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	~~2~~	~~0~~	~~0~~	~~1~~
P1	~~4~~	~~2~~	~~2~~	~~1~~
P2	~~2~~	~~1~~	~~0~~	~~3~~
P3	~~1~~	~~3~~	~~1~~	~~2~~
P4	~~1~~	~~4~~	~~3~~	~~2~~

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
2	2	2	1
4	2	2	2
5	5	3	4
6	9	6	6
10	11	8	7
12	12	8	10

PROCESS	NEED			
	A	B	C	D
P0	2	2	1	1 ✔
P1	1	0	3	1 ✔
P2	0	2	1	3 ✔
P3	0	1	1	2 ✔
P4	2	2	3	3 ✔

3. If request from P1 arrives for (1,1,0,0). Can request be immediately granted?

System pretends that the resource is allocated and check for safe state (no deadlock) if allocated. **(Uses Safety Algorithm)**

Find Safe Sequence:

Safe Sequence: <P0, P3, P4, P1, P2>

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	~~2~~	~~0~~	~~0~~	~~1~~
P1	~~4~~	~~2~~	~~2~~	~~1~~
P2	~~2~~	~~1~~	~~0~~	~~3~~
P3	~~1~~	~~3~~	~~1~~	~~2~~
P4	~~1~~	~~4~~	~~3~~	~~2~~

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
2	2	2	1
4	2	2	2
5	5	3	4
6	9	6	6
10	11	8	7
12	12	8	10

PROCESS	NEED			
	A	B	C	D
P0	2	2	1	1 ✔
P1	1	0	3	1 ✔
P2	0	2	1	3 ✔
P3	0	1	1	2 ✔
P4	2	2	3	3 ✔

3. If request from P1 arrives for (1,1,0,0). Can request be immediately granted?

System pretends that the resource is allocated and check for safe state (no deadlock) if allocated. **(Uses Safety Algorithm)**

Find Safe Sequence:

Safe Sequence: <P0, P3, P4, P1, P2>

GRANTS REQUEST IMMEDIATELY

Total No. of Instances: (Allocated + Available)

A	B	C	D
12	12	8	10

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	2	0	0	1
P1	3	1	2	1
P2	2	1	0	3
P3	1	3	1	2
P4	1	4	3	2

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
3	3	2	1

PROCESS	NEED			
	A	B	C	D
P0	2	2	1	1
P1	2	1	3	1
P2	0	2	1	3
P3	0	1	1	2
P4	2	2	3	3

4. If request from P4 arrives for (0,0,2,0). Can it be immediately granted?

Uses Resource Request Algorithm

Step 1: Check **REQUEST(P4) <NEED(P4)** → (0,0,2,0) < (2,2,3,3) → Yes

Step 2: Check **REQUEST(P4) < =AVAILABLE** → (0,0,2,0) < = (3,3,2,1) → Yes

So, the request from P4 for (0,0,2,0) can be granted

Step 3: But, the System will not immediately allocate the resource. Rather, it pretends that the resource is allocated and check for safe state (no deadlock) if allocated. **(Uses Safety Algorithm)**

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	2	0	0	1
P1	3	1	2	1
P2	2	1	0	3
P3	1	3	1	2
P4	1	4	~~3~~ 5	2

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
3	3	~~2~~ 0	1

Updated System Snapshot

PROCESS	NEED			
	A	B	C	D
P0	2	2	1	1
P1	2	1	3	1
P2	0	2	1	3
P3	0	1	1	2
P4	2	2	~~3~~ 1	3

4. If request from P4 arrives for (0,0,2,0). Can it be immediately granted?

System pretends that the resource is allocated and check for safe state (no deadlock) if allocated. **(Uses Safety Algorithm)**

Update NEED (P4) = NEED (P4) – REQUEST (P4)

AVAILABLE = AVAILABLE – REQUEST (P4)

ALLOCATION (P4) = ALLOCATION (P4) + REQUEST (P4)

Deadlock Avoidance Algorithms

Example 2

Banker's Algorithm

NEED Matrix

$NEED = MAX_i - ALLOC_i$

PROCESS	ALLOCATION			
	A	B	C	D
P0	2	0	0	1
P1	3	1	2	1
P2	2	1	0	3
P3	1	3	1	2
P4	1	4	~~3~~ 5	2

MAX			
A	B	C	D
4	2	1	2
5	2	5	2
2	3	1	6
1	4	2	1
3	6	6	5

AVAILABLE			
A	B	C	D
3	3	~~2~~ 0	1

Updated System Snapshot

PROCESS	NEED			
	A	B	C	D
P0	2	2	1	1
P1	2	1	3	1
P2	0	2	1	3
P3	0	1	1	2
P4	2	2	~~3~~ 1	3

4. If request from P4 arrives for (0,0,2,0). Can it be immediately granted?

System pretends that the resource is allocated and check for safe state (no deadlock) if allocated. **(Uses Safety Algorithm)**

Find Safe Sequence:

Safe Sequence: No Safe Sequence

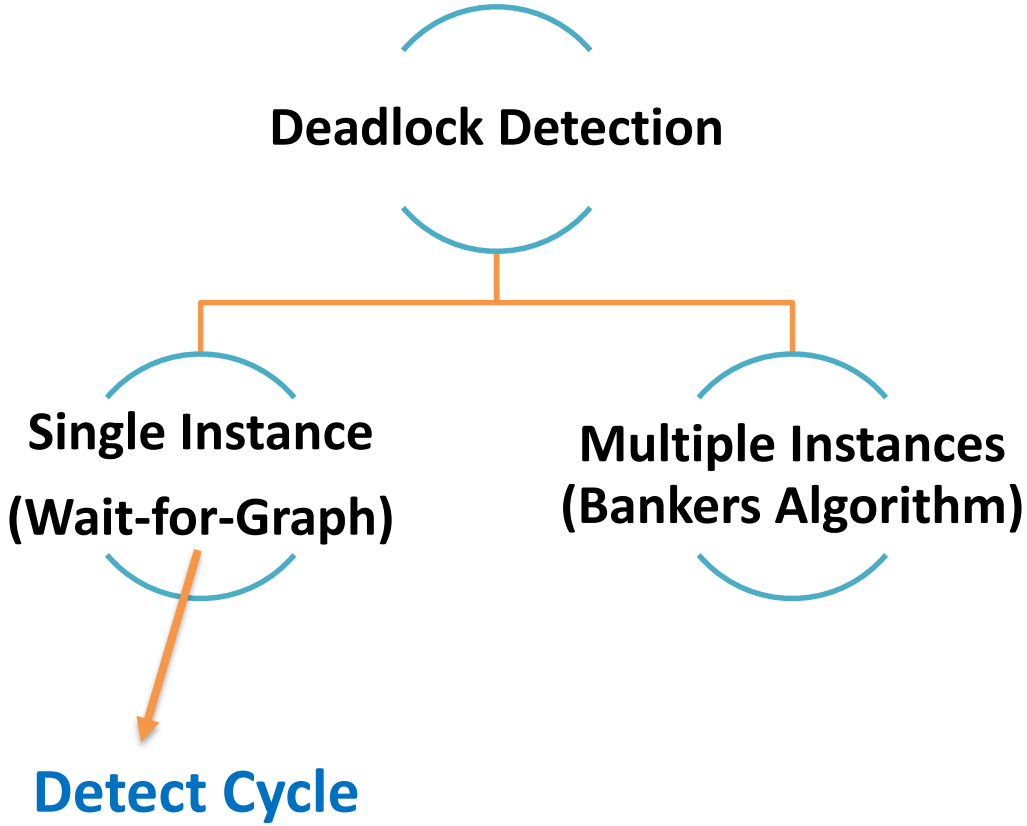
REQUEST NOT GRANTED

Deadlock Avoided

Deadlock Detection and Recovery

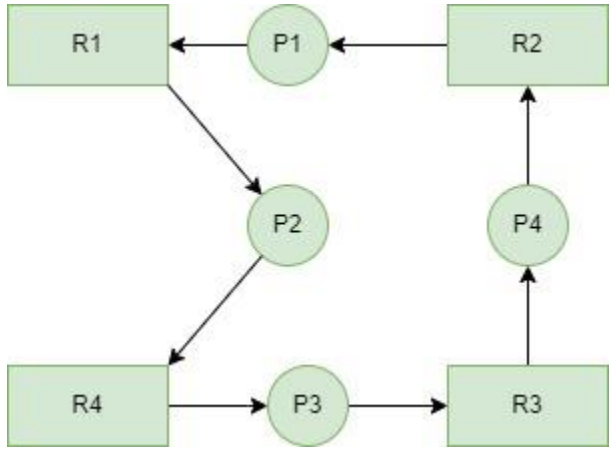
- Allow the system to enter into deadlock state
- Deadlock Detection Techniques
    1. Wait-for-graph (Single instance)
    2. Banker's Algorithm (Multiple Instances)

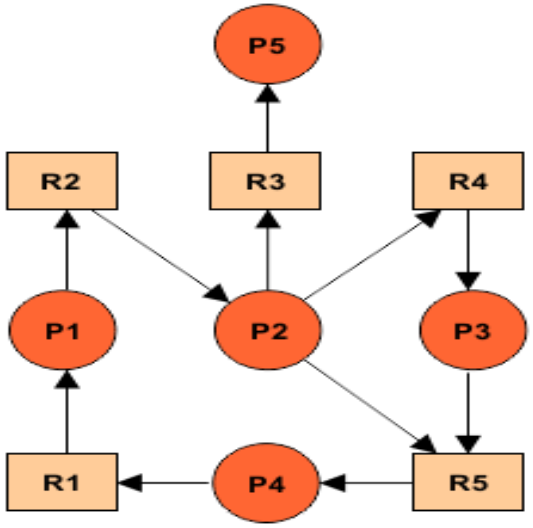
- Recovery Techniques

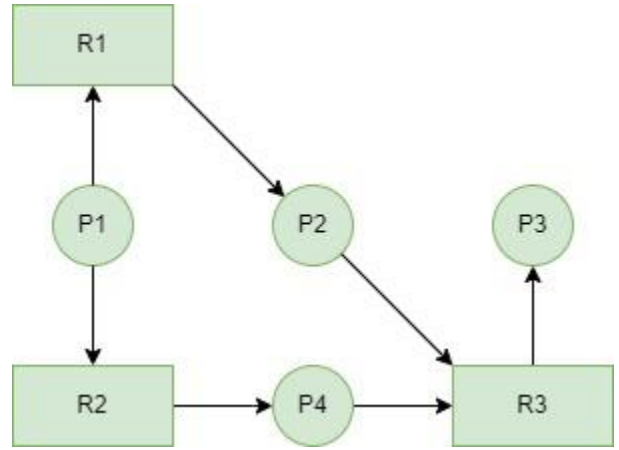


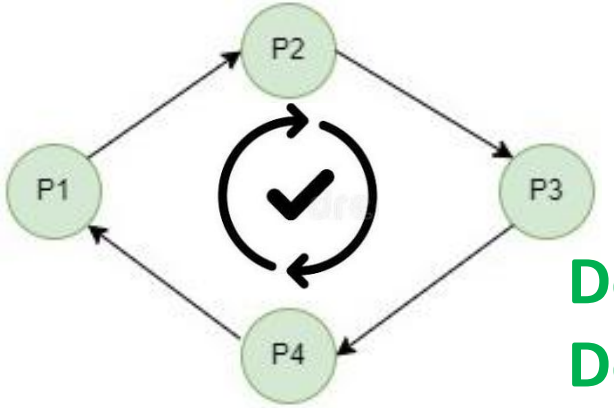
Deadlock Detection and Recovery

Deadlock Detection with Wait-for-graph (Single instance)

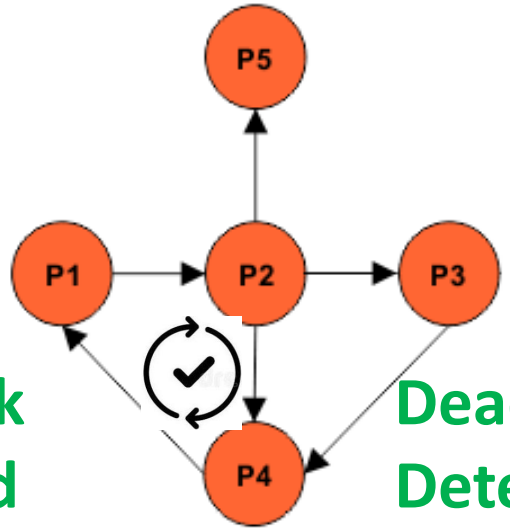




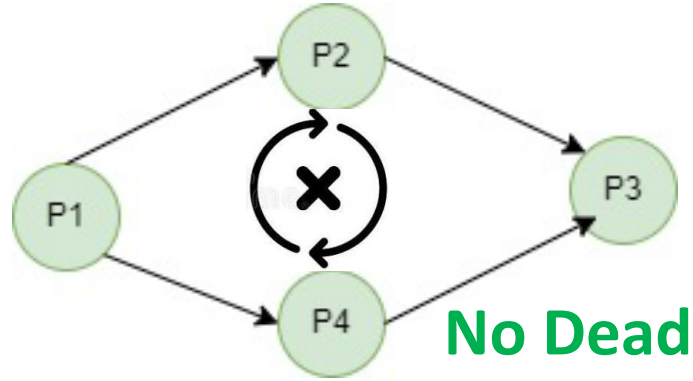




Deadlock Detected



Deadlock Detected



No Deadlock

Deadlock Detection and Recovery

Deadlock Detection with Bankers Algorithm (Multiple instance)

1. Let ***Work*** and ***Finish*** be vectors of length ***m*** and ***n***, respectively Initialize:
   a) ***Work* = *Available***
   b) For ***i* = 1,2, …, *n***,
      if ***Allocation***$_i \neq$ **0**, then ***Finish*[i] = *false***;
      otherwise, ***Finish*[i] = *true***

2. Find an index ***i*** such that both:
   a) ***Finish*[*i*] == *false***
   b) ***Request***$_i \leq$ ***Work***

   If no such ***i*** exists, go to step 4

3. ***Work* = *Work* + *Allocation***$_i$
   Finish*[*i*] = *true
   go to step 2

4. If ***Finish[i] == false***, for some ***i***, $1 \leq i \leq n$, then the system is in deadlock state.

   Moreover, if ***Finish*[*i*] == *false***,
   then T_i is deadlocked

Algorithm requires an order of O(m x n^2) operations to detect whether the system is in deadlocked state

Deadlock Detection and Recovery

Deadlock Detection with Bankers Algorithm (Multiple instance)

Example

	Allocation	*Request*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

0	1	0	P_0
3	1	3	P_2
5	2	4	P_3
7	2	4	P_1
7	2	6	P_4

Five processes P_0 through P_4;
Three resource types:
A (7 instances), *B* (2 instances), and *C* (6 instances)

Total

A B C

7 2 6

Sequence <P_0, P_2, P_3, P_1, P_4> will result in ***Finish[i] = true*** for all ***i***

Step1: All Processes have **finish[i] = false**, Since ***Allocation***$_i \neq$ **0.** (i.e., NOT COMPLETED and DIDN'T RELEASE ALLOCATED RESOURCES)

Step 2: Find a process with ***Finish*[*i*] == *false* && *Request*$_i \leq$ *Work.***

Step3: If such process exists, ***Work = Work + Allocation***$_i$ ***Finish*[*i*] = *true* ;** go to step 2

Step4: If no such process, If ***Finish[i] == false***, for some ***i***, $1 \leq i \leq n$, then the system is in deadlock state.
(i.e., NO PROCESS IS ABLE TO COMPLETE – THAT RESULTS IN DEADLOCK)

Hence, NO DEADLOCK

Deadlock Detection and Recovery

Deadlock Detection with Bankers Algorithm (Multiple instance)

Example

	Allocation	*Request*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	**0 0 1**	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

0	1	0	P_0

If P2 requests for an additional instance of C, then???????

Total

A B C

7 2 6

Sequence ***stops after P0 and*** will result in ***Finish[i] = false*** for all ***P1, P2, P3 and P4.***

Step1: All Processes have **finish[i] = false**, Since ***Allocation***$_i \neq$ **0. (**i.e., NOT COMPLETED and DIDN'T RELEASE ALLOCATED RESOURCES)

Step 2: Find a process with ***Finish*[*i*] == *false* && *Request*$_i \leq$ *Work.***

Step3: If such process exists, ***Work = Work + Allocation***$_i$ ***Finish*[*i*] = *true ;*** go to step 2

Step4: If no such process, If ***Finish[i] == false***, for some ***i***, $1 \leq i \leq n$, then the system is in deadlock state. (i.e., NO PROCESS IS ABLE TO COMPLETE – THAT RESULTS IN DEADLOCK)

Hence, DEADLOCK DETECTED

Deadlock Detection and Recovery

Deadlock Detection Algorithm Usage

- When, and how often, to invoke depends on:
  - How often a deadlock is likely to occur?
  - How many processes will need to be rolled back?
    - one for each disjoint cycle

- If detection algorithm is invoked arbitrarily,
  - there may be many cycles in the resource graph
  - would not be able to tell which of the many deadlocked threads “caused” the deadlock.

Deadlock Detection and Recovery

Deadlock Recovery - Process Termination

- Abort all deadlocked threads
- Abort one process at a time until the deadlock cycle is eliminated
- In which order should we choose to abort?
  - Priority of the thread
  - How long has the thread computed, and how much longer to completion
  - Resources that the thread has used
  - Resources that the thread needs to complete
  - How many threads will need to be terminated
  - Is the thread interactive or batch?

Deadlock Recovery - Resource Preemption

- **Selecting a victim** – minimize cost
- **Rollback** – return to some safe state, restart the thread for that state
- **Starvation** – same thread may always be picked as victim, include number of rollback in cost factor

BCSE303L - Operating Systems



Module 3 – Scheduling

Module:3	Scheduling	9 hours
Processes Scheduling - CPU Scheduling: Pre-emptive, non-pre-emptive - Multiprocessor scheduling – Deadlocks - Resource allocation and management - Deadlock handling mechanisms: prevention, avoidance, detection, recovery.		